Errata Notice:
ARM PrimeCell
PL011 Universal Asynchronous Receiver Transmitter

IP Products Division

Document Part Number	PL011-DC-11001
Document Reference	PL011-DEI-0000
Date of Issue:	12 December 01
Author	Margaret Rugira
Authorised by:	Michael Staplehurst

Abstract

This document details known errata for the ARM PrimeCell PL011 Universal Asynchronous Receiver Transmitter (PL011)

Keywords

PrimeCell PL011 UART

Distribution list

Name	Function
Michael Staplehurst	IP Products Division Operations Manager
Martyn Jones	PrimeCell Technical Lead
Katherine Wright	Technical Lead
John Slater	PrimeCell Product Manager

1 ABOUT THIS DOCUMENT

1.1 Change control

1.1.1 Change history

This refers to the revision changes to the Errata document

Issue	Date	By	Change
1.0	12th December 01	Margaret Rugira	First Draft

1.2 Errata List

This table summarises the errata notices detailed

Release: Release revision number affected

Section: Section in the document where the issues are described

Status: The status conditions are:

Open: Issue is current

Closed: Issue resolved in subsequent release

Severity: The severity conditions are:

A **Low** level of severity indicates that there is no direct impact to the silicon implementation. Examples are:

- No change to RTL required
- Documentation change
- Test pattern change
- Synthesis Scripts change

A **Medium** level of severity indicates that there is a minor impact to the silicon implementation, which can be overcome by modifications detailed in the errata notes. Examples are:

- A specific sequence of events not adhered to.
- Ambiguous or unexpected operation of the device.

A **High** level of severity indicates that there is a major impact to the silicon implementation, which cannot be overcome without modifications detailed in the errata notes. Examples are:

- Non conformance to the device specification.

Affected: Deliverables impacted by the issue

Description: Issue concerning Errata

Release	Section	Status	Severity	Affected	Description
1v0	2.1	Closed	HIgh	Some RTL files	Fractional Baud Rate Divisor Does Not Function With Some Divisor Values
1v0	2.2	Closed	High	Some RTL files	DMA signals not synchronised to UART PCLK
1v0	2.3	Closed	Medium	IrDA Low Power Counter	IrDA low power counter consumes power when IrDA mode is disabled.
1v0	2.4	Open	Low	apbslave_tb.v	Two Parameters Instead of Three Are Being Passed To Clockgen
1v0	2.5	Closed	Medium	Some RTL files	Some redundant logic in the Integration test logic
1v0	2.6	Closed	Low	DDES0000A	Design Manual Documentation Error
1v1	3.1	Closed	Medium	Some RTL files	UART appears to ignore disable function
1v1	3.2	Closed	High	UartTXCtrl.v , UartTXFIFO.v	Uart CharTxComp Signal Is Not Synchronised
1v1	3.3	Closed	Low	DDES0000B	Design Manual Documentation Error
1v1	3.4	Open	Low	apbslave_tb.v	Two Parameters Instead of Three Are Being Passed To Clockgen
1v3	4.1	Open	Low	apbslave_tb.v	Two Parameters Instead of Three Are Being Passed To Clockgen

2 PL011 RELEASE 1V0 - ERRATA DETAILS

2.1 Fractional Baud Rate Divisor Does Not Function With Some Divisor Values

2.1.1 Summary

The original counter used the fractional part to select between the output of two separate 6-bit counters. These were /n, and /(n+1). If a value f is programmed into the UARTFBRD register then out of the 64 pulses generated, f comes from the (n+1) counter, and (64-f) comes from the n counter. All counters were clocked off UARTCLK. It was found that when f was an integer fraction of 64 (e.g.16) the switching could be synchronized with one of the counters and it was possible for pulses to be repeatedly lost. The resulting baud rate deviation errors could be up to about 25%.

The factor integer fraction of 64 is due to the fact that the 6-bit fractional part of the baud rate divisor (m), can be calculated by taking the fractional part of the required baud rate and multiplying it by and adding 0.5 to account for the rounding errors. It is this bit that more or less does not work as it should do.

2.1.2 Severity

High

The error occurs when the fractional divider function is utilized. The product works correctly if this function is not utilized. However, how to overcome and implement this is documented below.

2.1.3 Symptoms

Examples of errors encountered:

Whenever the fractional baud rate divider register (FBRD) is set to a value other than 0, the resulting baud rate is wrong, where as when FBRD is 0 the baud rate matches exactly what is expected

Examples:
IBRD = 16, FBRD = 18: resulting division factor is approximately 21
IBRD = 37, FBRD = 10: resulting division factor is approximately 38
IBRD = 16, FBRD = 0: resulting division factor is 16 (correct)
IBRD = 1, FBRD = 0: resulting division factor is 1 (correct)

The example:
IBRD = 16, FBRD = 18

has been simulated and the results are as described above (i.e. division factor of approximately 21).
However, the examples in the original test bench do seem to work (with approximately 2-3% error).

With a Uart clock of 80MHz and a required baud rate of 115,200:

UARTIBRD should be = 43 & UARTFBRD = 26.
The UART does not work correctly with these values.

However, with:
UARTIBRD = 43 & UARTFBRD = 27, the UART works correctly.

2.1.4 Affected Items

Uart.v/vhd

UartApbif.v/vhd

UartBaudCntr.v/vhd

UartRegBlock.v/vhd

2.1.5 Short term work around

The fractional baud rate divider register UARTFBRD mentioned on page 2-11 and 3-10 of the Technical Reference Manual (ARM DDI 0183A), used by the programmable baud generator should not be used and should be programmed to zero. Hence you should only use the integer register UARTIBRD.

Both these registers UARTFBRD and UARTIBRD are cleared on reset. Therefore it must be ensured that as the fractional baud rate generator is not being used, at no point during the working of the UART should there be a zero value in the integer UARTIBRD register. Otherwise no transmission or reception will take place (see note on page 3-10).

Some manual testing of different UART CLK / baud rate combinations has been carried out and the baud rate has been verified with 'scope as well as the reception of characters on our prospector board.

The following table shows the baud rate errors for a fixed input frequency.

UART CLK (hz) prog	Uart CLK measured (AV)	Baud	Pulse width (Average) (s)	Baud (Measured)	%error	Character Transmission ok
7.3728.0e+6	7.37382.0e+6	1200	1.56228E-04	1200.16898	-0.01	ok 80%
	7.37382.0e+6	2400	7.81139E-05	2400.34104	-0.01	ok 80%
	7.37382.0e+6	9600	1.95218E-05	9604.64711	-0.05	ok 80%
	7.37382.0e+6	14400	1.30187E-05	14402.35968	-0.02	ok 80%
	7.37382.0e+6	19200	9.76390E-06	19203.39209	-0.02	ok 80%
	7.37382.0e+6	38400	4.88168E-06	38408.90841	-0.02	ok 100%
	7.37382.0e+6	57600	3.25426E-06	57616.78538	-0.03	ok 100%
	7.37382.0e+6	115200	1.62691E-06	115249.15330	-0.04	ok 100%
5.00000E+06	5.00066E+06	1200	1.56233E-04	1200.13057	-0.01	ok 70%
		2400	7.81062E-05	2400.57768	-0.02	ok 100%
		9600	1.94750E-05	9627.72786	-0.29	ok 80%
		14400	1.30153E-05	14406.12203	-0.04	ok 100%
		19200	9.74510E-06	19240.43878	-0.21	ok 100%
		38400	4.86330E-06	38554.06823	-0.40	ok 100%
		115200	1.62854E-06	115133.80083	0.06	ok 99%
		230400	4.06340E-07	461436.23566	-100.28	total garbage

In summary, to ensure that no further problems are encountered, only use the programmable 16-bit integer part, loaded through the UARTIBRD register, page 3-10.

However, should the fractional baud rate register be required, then a spreadsheet has been produced that has a series of macro's built in, enabling the integer and fractional values to be calculated for a given input frequency and baud-rate requirement. The spreadsheet models the error component known to exist in the Rel1v0 release enabling values to be picked that have low/minimal error rates.

A spreadsheet for calculating the values that can be used in the fractional divider is available from ARM Support: email support-primecell@arm.com

2.1.6 Proposed Corrective action

A new release of the PL011, Rel1v1 is available in which the Fractional Baud Rate divider has been redesigned. The new implementation of the fractional baud rate divider only uses one 16-bit counter instead of the previous 2.

The legal values of the divisor have changed with the new implementation and logic has been introduced, such that the line control register (UARTLCR_H) and the baud rate divider registers (UARTIBRD and UARTFBRD) only get updated once the current character has completed.

2.2 DMA Signals Not Synchronised to UART PCLK

2.2.1 Summary

The UARTTXDMACLR and UARTRXDMACLR from the DMA controller are not being synchronised to the UARTPCLK clock domain.

The DMA capable peripheral requires additional external synchronisation logic on the DMACLR and DMATC signals if the peripherals do not use the same clock as the DMA controller. This synchronisation logic should ideally be in the peripheral itself.

This synchronisation logic can be either always used, or enabled and disabled using a register bit.

2.2.2 Severity

High

2.2.3 Symptoms

No symptoms available to demonstrate this issue as problem was discovered through observation of the code.

2.2.4 Affected Items

Uart.v/vhd

UartApif.v/vhd

UartDMA.v/vhd

UartRegBlock.v/vhd

UartTest.v/vhd

2.2.5 Short Term Workaround

There is no short-term workaround.

2.2.6 Proposed Corrective action

Synchronisation logic has been inserted in the UARTTXDMACLR and UARTRXDMACLR paths in the Rel1v1 release of the UART.

2.3 IrDA low power counter consumes power when IrDA mode is disabled.

2.3.1 Summary

An eight bit counter within the IrDA section continues counting even when the IrDA mode is disabled. The counting action should be disabled whenever the IrDA mode is disabled.

2.3.2 Severity

Medium

2.3.3 Symptoms

No symptoms available to demonstrate this issue as problem was discovered through simulations.

2.3.4 Affected Items

UartBaudCntr.v/vhd

UartIrDa.v/vhd

UartSynctoUCLK.v/vhd

UartTest.v/vhd

2.3.5 Short Term Workaround

There is no short-term workaround.

2.3.6 Proposed Corrective action

Changes have been made so that the IrDa low power counter does not count when operating in Uart mode, i.e the counter will only count when the Uart is running in normal IrDa mode and low power IrDA mode.

2.4 Two Parameters Instead of Three Are Being Passed To Clockgen

2.4.1 Summary

The module clockgen.v requires three parameters to be passed to it, but in its instantiation, only two are being passed, resulting in one value being held constant.

2.4.2 Severity

Low

Mismatch does not affect functionality of the device.

2.4.3 Symptoms

There will be a parameter mismatch between CLOCKGEN module call in apbslave_tb.v and the CLOCKGEN module definition in clockgen.v.

2.4.4 Affected Items

apbslave_tb.v file

2.4.5 Short Term Workaround

Change, within the apbslave_tb.v file:

```
CLOCKGEN #(`Tclkh, `Tclkl) U_CLOCKGEN
(.PCLK(I_CLK));
```

to:

```
CLOCKGEN #(`Tclks, `Tclkh, `Tclkl) U_CLOCKGEN
(.PCLK(I_CLK));
```

This will resolve the mismatch.

2.4.6 Proposed Corrective action

This was first discovered in Rel1v1 and will be corrected in the next major release of the UART

2.5 Some redundant logic in the Integration test logic

2.5.1 Summary

1) UARTITIP is defined as an 8-bit register but only bits 7 and 6 are used. It should be defined as UARTITIP [7:6].

2) The primary input signals which are read via the UARTITIP register should be taken from before the

synchronisers, as they will only ever be read in a test mode.

2.5.2 Severity

Medium

Does not affect functionality of the device.

2.5.3 Symptoms

No symptoms available to demonstrate this issue.

2.5.4 Affected Items

Uart_Integ.c
Uart.h

2.5.5 Short Term Workaround

There is no short-term solution to this.

2.5.6 Proposed Corrective action

The redundant logic was removed in a subsequent release of the device, REL1v1.

The primary input signals are now read prior to their synchronisation logic.

2.6 Design Manual Documentation Error

2.6.1 Summary

An equation did not match the state diagram in Figure 26 of the Design Manual.

2.6.2 Severity

Low

These errors are issued as documentation errata and do not affect the operation or verification of the device.

2.6.3 Symptoms

Typing mistake.

The state diagrams in the design manual do not match the equations implemented in the code.

The equation in the design manual is:

((CTSEn + CTSSyncUARTCLK') * CTSEn) + TXDataAvlblSync*Baud16*AbortTransmit'

The equation implemented in the code is:

((CTSEn * CTSSyncUARTCLK') + CTSEn') * TXDataAvlblSync*Baud16*TXEnable*AbortTransmit'

2.6.4 Affected Items

PL0111DDES0000A.pdf Page 47

2.6.5 Short term work around

Published as documentation errata.

2.6.6 Proposed Corrective action

This has been corrected in Rev C of the design manual with the new release of the UART REL1v3.

3 PL011 RELEASE 1V1 - ERRATA DETAILS

3.1 Uart Ignores the Disable Function

3.1.1 Summary

Whenever the FIFO is disabled during a transmit, the UART should continue to process the current data byte, and then pause. However, the Uart appears to ignore the disable function and continues transmitting under the following conditions:

- Under hardware control and multiple words in the transmit FIFO, the UART transmits the most recent character repeatedly if it was disabled before the character ended; thus the hardware flow control overrides the enable signal;
- When the UART was disabled before a character ended and there was a single word in the FIFO. The current character finished transmission as expected. However, when the UART was re-enabled, the last character was retransmitted.

3.1.2 Severity

High

3.1.3 Symptoms

Repetition of characters or the last character transmitted may be seen.

3.1.4 Affected Items

UartTxCntl.v

3.1.5 Short term work around

1) For the case where the hardware flow control overrides the enable signal. This can be fixed using the following code:

   UartTxCntl.v Line 620

```
//
------------------------------------------------------------------
// For Flow Control need to check whether it is still okay to
send
// further data (CTSEn = '1' and nCTSSyncUARTCLK = '0'),
provided
// that TXEnable is still true
// Otherwise normal operation resumes.
//
-------------------------------------------------------------------
if (CTSEn == 1'b1) begin
if ((nCTSSyncUARTCLK == 1'b0) && (TXEnable == 1'b1))
```

For the case when the UART is disabled during the transmission of a character, then re-enabled after that character completes; there is no suitable workaround without affecting the rest of the UART verification code.

3.1.6 Proposed Corrective action

Both issues have been corrected in the Rel1v3 release of the UART.

3.2 Uart CharTxComp Signal Is Not Synchronised

3.2.1 Summary

The CharTxComp signal from UartTXCtrl.v (uartclock domain) needs to be synchronised before being used in UartTXFIFO.v (pclk domain).

3.2.2 Severity

High

There are issues with using multiple synchronised signals to control state machines in the TxFifo if the order of signals from UARTCLK are significant.

3.2.3 Symptoms

Characters may be retransmitted.

3.2.4 Affected Items

UartTXCtrl.v and UartTXFIFO.v

3.2.5 Short term work around

There is no short-term solution.

3.2.6 Proposed Corrective action

The code has been changed by adding synchronisation logic to this signal in the new release of the UART (REL1v3).

3.3 Design Manual Documentation Error

3.3.1 Summary

An equation did not match the state diagram in Figure 28 of the Design Manual.

3.3.2 Severity

Low

These errors are issued as documentation errata and do not affect the operation or verification of the device.

3.3.3 Symptoms

Typing mistake.

The state diagrams in the design manual do not match the equations implemented in the code.
The equation in the design manual is:

((CTSEn + CTSSyncUARTCLK') * CTSEn) + TXDataAvlblSync*Baud16*AbortTransmit'

The equation implemented in the code is:

((CTSEn * CTSSyncUARTCLK') + CTSEn') * TXDataAvlblSync*Baud16*TXEnable*AbortTransmit'

3.3.4 Affected Items

PL011DDES0000B.pdf Page 43

3.3.5 Short term work around

Published as documentation errata.

3.3.6 Proposed Corrective action

This has been corrected in Rev C of the design manual with the new release of the UART REL1v3.

3.4 Two Parameters Instead of Three Are Being Passed To Clockgen

3.4.1 Summary

The module clockgen.v requires three parameters to be passed to it, but in its instantiation, only two are being passed, resulting in one value being held constant.

3.4.2 Severity

Low

Mismatch does not affect functionality of the device.

3.4.3 Symptoms

There will be a parameter mismatch between CLOCKGEN module call in apbslave_tb.v and the CLOCKGEN module definition in clockgen.v.

3.4.4 Affected Items

apbslave_tb.v file

3.4.5 Short Term Workaround

Change, within the apbslave_tb.v file:

```
CLOCKGEN #(`Tclkh, `Tclkl) U_CLOCKGEN
(.PCLK(I_CLK));
```

to:

```
CLOCKGEN #(`Tclks, `Tclkh, `Tclkl) U_CLOCKGEN
(.PCLK(I_CLK));
```

This will resolve the mismatch.

3.4.6 Proposed Corrective action

This will be resolved in the next major release of the UART.

4 PL011 RELEASE 1V3 - ERRATA DETAILS

4.1 Two Parameters Instead of Three Are Being Passed To Clockgen

4.1.1 Summary

The module clockgen.v requires three parameters to be passed to it, but in its instantiation, only two are being passed, resulting in one value being held constant.

4.1.2 Severity

Low

Mismatch does not affect functionality of the device.

4.1.3 Symptoms

There will be a parameter mismatch between CLOCKGEN module call in apbslave_tb.v and the CLOCKGEN module definition in clockgen.v.

4.1.4 Affected Items

apbslave_tb.v file

4.1.5 Short Term Workaround

Change, within the apbslave_tb.v file:

```
CLOCKGEN #(`Tclkh, `Tclkl) U_CLOCKGEN
(.PCLK(I_CLK));
```

to:

```
CLOCKGEN #(`Tclks, `Tclkh, `Tclkl) U_CLOCKGEN
(.PCLK(I_CLK));
```

This will resolve the mismatch.

4.1.6 Proposed Corrective action

This will be resolved in the next major release of the UART.